

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 4

Выполнил:

Баженов Алексей

Группа К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделить монолитное бэкенд-приложение бронирования ресторанов на 4 микросервиса с собственными базами данных, реализовать API Gateway и межсервисное взаимодействие

Ход работы

1. Исходный монолит

Структура монолита до разделения:

Одна БД со всеми 7 таблицами. Прямые связи через FK.

Исходный монолит был организован так: в папке controllers размещались обработчики HTTP-запросов, разделённые по доменным областям (аутентификация, пользователи, рестораны, бронирования, отзывы); в папке models находились TypeORM-энтити, описывающие все таблицы базы данных и связи между ними через FK; вспомогательные модули (типа JWT-верификация, хэширование паролей) лежали в middlewares и utils. Единая точка входа app.ts запускала Express-сервер, подключала все контроллеры и устанавливала соединение с единственной БД

2. Целевая архитектура

Микросервис	Порт	База данных	Таблицы	Функционал
auth-user-service	8001	auth_db	users	Регистрация, логин, JWT, профиль пользователя
restaurant-service	8002	restaurant_db	restaurants, tables, menu_items, restaurant_photos	Управление каталогом ресторанов, столиками, меню, фотографиями, поиск и фильтрация
reservation-service	8003	reservation_db	reservations	Создание и отмена бронирований, проверка доступности столиков

review-service	8004	review_db	reviews	Отзывы, рейтинги
----------------	------	-----------	---------	------------------

Internal связи:

- reservation → auth-user: проверка пользователя
- reservation → restaurant: проверка столика
- review → restaurant: обновление рейтинга

3. Действия по разделению

3.1 Создание структуры папок

Целевая структура построена по принципу database-per-service: каждый микросервис вынесен в отдельную директорию внутри папки services, содержит собственный package.json, TypeORM-конфигурацию и полный набор исходного кода (контроллеры, модели, утилиты).

Папка gateway содержит API Gateway, который маршрутизирует входящие запросы на соответствующие сервисы и проверяет JWT. Файл docker-compose.yml описывает запуск всех 9 контейнеров (4 БД, 4 сервиса, gateway), а .env хранит секреты

3.2 Копирование и адаптация кода

Для каждого сервиса:

1. Скопированы нужные контроллеры и модели
2. Удалены FK— заменены на логические ссылки
3. Добавлены internal контроллеры для межсервисных вызовов
4. Добавлены HTTP-клиенты для вызова других сервисов
5. Настроены отдельные подключения к БД через TypeORM

3.3 Реализация API Gateway

Файл gateway/server.js:

- Проксирует запросы на соответствующие сервисы
- Проверяет JWT токены для защищённых маршрутов

3.4 Docker Compose

Конфигурация из 9 контейнеров (4 БД + 4 сервиса + gateway). Все переменные вынесены в .env

4. Результат

После выполнения работы:

1. Монолит разделён на 4 микросервиса
2. Каждый сервис имеет свою БД
3. Реализован API Gateway
4. Настроены internal HTTP вызовы между сервисами
5. Все сервисы запускаются через docker-compose
6. Swagger доступен на каждом сервисе: <http://localhost:800X/docs>

Вывод

Монолит успешно разделён на 4 микросервиса с отдельными БД. Реализовано синхронное HTTP взаимодействие через Internal API. Разработан Gateway и Docker Compose для запуска всей системы. Документация OpenAPI доступна через Swagger на каждом сервисе.